

TAKE-HOME ASSIGNMENT

ScreenIQ — AI-Powered Candidate Screener

Full-Stack Developer · React / Next.js · Django · AI Integration

Duration	Level	Deadline	Submission
2 – 3 Hours	Mid – Senior	72 Hours	GitHub

1. Overview

You are building **ScreenIQ**, a lightweight internal tool that helps HR teams screen job applicants using AI. The application allows a user to paste a job description and a candidate's resume, receive an AI-generated shortlisting score with reasoning, and manage a list of past screenings.

You will build both the Django REST API backend and the Next.js frontend. The AI integration can use any provider of your choice — OpenAI, Anthropic, or Google Gemini.

i You are encouraged to use any tools, libraries, or references you normally would on the job.

What matters is your reasoning, your decisions, and your ability to explain them.

Tech Stack

- Backend: Django 4.x + Django REST Framework
- Frontend: Next.js 14+ with App Router
- Database: PostgreSQL
- AI: OpenAI / Anthropic / Gemini (your choice)
- Auth: JWT (djanoorestframework-simplejwt recommended)

2. Part A — Django Backend

You are provided with a starter `views.py` file below. It has issues. Your first task is to identify and fix them before extending the code.

```
# views.py — starter code
```

```
from rest_framework.views import APIView
```

```
from rest_framework.response import Response
```

```
from rest_framework import status
```

```
from .models import Application
```

```
from .serializers import ApplicationSerializer
```

import openai
class ScreenCandidateView(APIView):
def post(self, request):
job_desc = request.data['job_description']
resume = request.data['resume']
response = openai.ChatCompletion.create(
model="gpt-4",
messages=[{"role": "user", "content": f"Score this resume 1-10: {resume}"}]
)
score = response.choices[0].message.content
app = Application.objects.create(
job_description=job_desc,
resume=resume,
ai_score=score,
created_by=request.user
)
return Response(ApplicationSerializer(app).data,
status=status.HTTP_200_OK)

TASK A-1 Debug & Fix the Starter Code

The code above has multiple issues. Find them all, fix them, and for each fix write a short comment in the code explaining what was wrong and why your fix is correct.

Also add a "Bugs Fixed" section to your README listing each issue and your explanation.

TASK A-2 Build the AI Screening Prompt

The current prompt is too simple and will produce unreliable results. Design a proper prompt that:

- Uses both the job description and the resume
- Returns a score from 1 to 10
- Returns 3 concise bullet-point reasons for the score

In your README, write a short paragraph explaining the design decisions you made for this prompt.

TASK A-3 Find and Fix the Security Vulnerability

The model and view below contain a security vulnerability. Identify it, fix it, and explain it in your README.

```
class Application(models.Model):
    job_description = models.TextField()
    resume         = models.TextField()
    ai_score       = models.CharField(max_length=10)
    created_by     = models.ForeignKey(User,
on_delete=models.CASCADE)
    created_at     = models.DateTimeField(auto_now_add=True)

class ApplicationListView(APIView):
    permission_classes = [IsAuthenticated]
    def get(self, request):
        apps = Application.objects.all()
        return Response(ApplicationSerializer(apps, many=True).data)
```

3. Part B — Next.js Frontend

TASK B-1 Screening Form — /screen

Build a page at /screen with:

- A textarea for the Job Description
- A textarea for the Candidate Resume
- A Submit button that calls your Django API and displays the AI score and reasons

Choose whichever state management approach you think is appropriate for this use case, and write one paragraph in your README explaining why you chose it.

TASK B-2 Applications Dashboard — /dashboard

Build a page at /dashboard showing a table of past screenings. Columns should include:

- Candidate name (derived from the resume or manually entered)
- AI Score (colour-coded: red below 5, amber 5–7, green above 7)
- Date screened and a View detail button

The table must handle 500+ rows without lag. Implement either server-side pagination or virtual scrolling, and explain the tradeoff in your README.

TASK B-3 Edge Case — Inconsistent AI Output

The AI API sometimes returns the score as a decimal (e.g. 7.3) and sometimes as a word (e.g. "Seven"). Handle both cases gracefully so the UI always renders a usable score.

Document where you decided to handle this normalisation (frontend vs backend) and why.

4. Part C — AI Integration

TASK C-1 Stream the AI Response

Instead of waiting for the full AI response before displaying anything, stream the output progressively to the frontend so the score and bullet points appear as they are generated.

You may use Server-Sent Events, Django Channels, or Next.js route handlers. Document which approach you chose and why.

TASK C-2 Written Answer — Bias Detection (No Code Required)

HR has raised a concern: the AI might score candidates differently based on their name, university, or location rather than their actual skills.

In 200–300 words, answer: How would you detect whether this bias is present, and what steps would you take to reduce it? No code needed — this is a product and engineering thinking question.

Include this answer in a "Bias & Fairness" section of your README.

5. Submission Requirements

Please submit all of the following. Incomplete submissions will not be reviewed.

Item	Details
GitHub Repository	Public repo or invite our handle. Must include a working README with setup instructions.
README.md	Must include: setup guide, Bugs Fixed section, all decision justifications, Bias & Fairness answer.
.env.example	All required environment variable keys listed (no real values).
Tests (minimum 3)	Any framework. Test whatever you consider most critical. Explain briefly in README why you chose those tests.

6. Scoring Rubric

Your submission will be reviewed across the following dimensions. Code quality alone is not the primary measure — your reasoning, written explanations, and decision-making matter equally.

Area	Max Points
Bug identification & written explanation	20

Security vulnerability — detection & fix	15
AI prompt quality and reasoning (README)	20
Architecture decisions with justification	15
Edge case handling (AI output normalisation)	10
Bias detection written answer	10
Loom walkthrough clarity	10

Total: 100 points

A candidate with 60 points in code but strong written reasoning scores higher than one with 90 points in code and shallow explanations.

7. Ground Rules

- You may use any libraries, documentation, Stack Overflow, or AI tools you would normally use at work.
- Do not copy entire solutions from the internet and submit them as your own.
- If you make a decision that feels like a shortcut, note it in your README. We appreciate honesty and self-awareness.
- If you cannot complete a task, write what you would have done and why. Incomplete with good reasoning beats guessed code.